

/loop: automação segura de refatoração em lote

Pesquisa sobre o comando /loop do Claude Code CLI, motivada pela prática "O Engenheiro de Automação e o Tech Lead Microgerenciador": como refatorar 50 arquivos e criar testes de unidade sem ficar revisando linha a linha, evitando loops infinitos e sabendo argumentar a favor da automação em lote perante a liderança.

Fernando Melo · 9 de setembro de 2026

01 · O PROBLEMA

50 arquivos, zero tempo para vigiar cada linha

O time já usa o Claude para funções pequenas, mas o desafio agora é maior: 50 arquivos precisam de uma refatoração padronizada e de testes de unidade novos. O Tech Lead não tem tempo — e não quer que o time tenha — para acompanhar cada linha escrita pela IA.

A dúvida central: o que é o comando /loop, como ele evita que a IA fique "dando voltas em círculos" ao encontrar um erro, e se é realmente possível deixá-la trabalhando sozinha e só voltar quando o terminal avisar que terminou.

02 · O QUE É O /LOOP

Um "segundo turno" de desenvolvimento autônomo

O /loop é uma das ferramentas de automação mais poderosas do Claude Code CLI. Em suma, ele cria tarefas agendadas ou recorrentes baseadas em tempo/condições dentro da sessão do terminal, funcionando de forma parecida com um cron job.

Para o cenário dos 50 arquivos, isso significa poder delegar a tarefa inteira e tirar os olhos do processo, em vez de acompanhar cada alteração manualmente.

03 · TRAVAS CONTRA O LOOP INFINITO

Três mecânicas que impedem a IA de girar em círculos

O medo de deixar uma IA sozinha é ela encontrar um erro e gastar milhares de tokens tentando a mesma solução errada indefinidamente. O Claude Code mitiga isso com três controles:

1. Contexto limpo a cada ciclo (Git Worktrees)

Em vez de continuar uma conversa longa e confusa, a cada nova iteração o Claude cria um ambiente de trabalho isolado (git worktree), lê o estado mais recente do repositório do zero e aplica uma lógica focada — sem ficar preso a vieses de respostas anteriores.

2. Casamento com o comando /goal

Não basta dizer "refatore meu código": o ideal é combinar o loop com uma meta clara de sucesso, por exemplo "rode a refatoração arquivo por arquivo; o loop só acaba quando npm test passar sem erros e o linter estiver 100% verde". Ao atingir o objetivo, o próprio processo se encerra.

3. Travas de segurança e expiração

Os loops têm parâmetros estritos de encerramento por tempo ou iterações (ex.: --for 3h para expirar em 3 horas, ou travas automáticas de até 3 dias). Se o código quebrar de um jeito que a IA não consiga resolver, ela esgota as tentativas permitidas, para e registra o erro para análise posterior.

Sim, mas com uma estratégia de segurança

Com 50 arquivos, o volume é grande e o custo de tokens de API pode subir rápido se o escopo não estiver bem definido. O fluxo recomendado:

- 1 Alimente o CLAUDE.md: antes de soltar o loop, garanta que a raiz do projeto tenha um CLAUDE.md detalhado — o guia de estilo da equipe, com como os testes devem ser estruturados (framework, padrões de nomenclatura). O loop lê esse arquivo antes de cada alteração.
- 2 Faça um piloto com 1 ou 2 arquivos: rode um comando curto primeiro, só para ver como a IA se comporta.
- 3 Solte o loop em segundo plano: se o piloto der certo, rode a tarefa em lote e use `/background` para liberar o terminal. Depois, é só checar o progresso com `claude /loop --status`.

Do objetivo ao comando exato no terminal

Definir a condição de saída de forma objetiva e mensurável é o que separa um loop eficiente de um que se perde: se a meta for ambígua, a IA pode achar que terminou antes da hora; se for rígida demais sem critério de validação claro, ela pode travar. O padrão descrito na prática segue o ciclo Red-Green-Refactor autônomo, com submetas por arquivo e um teto de tentativas por unidade, que se traduz neste comando:

Comando executado no terminal:

```
claude /loop "Para cada um dos 50 arquivos pendentes: refatore  
seguindo o padrão, rode os testes. Se falhar, corrija em até 3  
tentativas. Só avance se os testes passarem." --until "npm test"
```

O parâmetro `--until` garante que o Claude só considera a tarefa encerrada quando o comando de teste retorna status code 0 (sucesso).

Custo de oportunidade: por que a IA não cansa

Refatorar 50 arquivos manualmente é repetitivo, entediante e propenso a erro humano — energia de desenvolvedores seniores que poderia estar em regras de negócio complexas. Dois argumentos centrais:

O paradoxo da tentativa e erro

Um humano gasta tempo no ciclo altera → testa → quebra → lê stack trace → altera de novo, e após a 4ª ou 5ª tentativa frustrada tende a perder o foco ou aceitar gambiarras. A IA repete o mesmo ciclo na velocidade da API, sem cansaço emocional, até encontrar a sintaxe que zera os erros.

Paralelização e "segundo turno" assíncrono

O maior desperdício em refatorações em lote é o tempo de máquina parado enquanto o dev espera os testes. Disparando o loop no fim do expediente, a IA processa o lote durante a noite; no dia seguinte o time recebe um Pull Request pronto para revisão, não um problema para resolver.

MÉTRICA	DEV HUMANO (MANUAL)	CLAUDE /LOOP (AUTÔNOMO)
Tempo por arquivo	30 a 45 minutos	1 a 3 minutos
Tempo total (50 arquivos)	~30 horas de engenharia	~2 horas de processamento assíncrono
Foco do engenheiro	Bloqueado na tarefa mecânica	Livre para tarefas estratégicas
Garantia de qualidade	Depende da atenção do dev	Travada via código pelo --until

07 · O ARGUMENTO MATADOR

Não eliminamos o olhar humano, eliminamos o trabalho braçal

"Nós não estamos eliminando o olhar humano; estamos eliminando o trabalho braçal. O time ainda vai revisar o código final no Pull Request, mas em vez de gastarmos 30 horas escrevendo e corrigindo testes na mão, gastaremos apenas 1 hora revisando o resultado de um agente que trabalhou de madrugada por uma fração do custo."

08 · PLANO DE PILOTO RECOMENDADO

Comece pequeno antes de soltar o loop nos 50 arquivos

- 1 Defina um escopo reduzido para o primeiro teste — recomendação de 3 a 5 arquivos (smoke test).
- 2 Estime um teto de gastos com tokens para aprovação do gestor antes de escalar para o lote completo.
- 3 Combine o comando de teste padrão do projeto (ex.: npm test) para que a IA valide o código modificado.
- 4 Depois do piloto, apresente as métricas de tempo economizado como base para o argumento de ROI junto à liderança.

Começar com um piloto reduzido valida a eficácia do /loop no ambiente do time sem colocar o projeto ou o orçamento de tokens em risco — e garante que os frutos da automação sejam colhidos com segurança.